

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

A derived class :

[Constructor]:

call a base class constructor via initializer list

[Inherits]:

all of the functions of the base class

[Inherits]:

all of the data of the base class

RedBall.h

```
1 #ifndef REDBALL_H
2 #define REDBALL_H
3
4 #include "Sphere.h"
5
6 class RedBall : public Sphere {
7     public:
8         RedBall(double r);
9
10
11
12
13
14     private:
15
16
17
18
19
20 }
21
22 #endif
```

RedBall.cpp

```
1 #include "RedBall.h"
2
3 namespace cs225 {
4
5     RedBall::RedBall(double r) :
6         Sphere(r) {
7         addProperty("Red");
8     }
9
10
11     // ...
12 }
13
14
15
16
17
18
19
20
21
22
```

Sphere.cpp

```
1 Sphere::print_1() {
2     cout << "Sphere" << endl;
3 }
4
5 Sphere::print_2() {
6     cout << "Sphere" << endl;
7 }
8
9 virtual Sphere::print_3() {
10     cout << "Sphere" << endl;
11 }
12
13 virtual Sphere::print_4() {
14     cout << "Sphere" << endl;
15 }
16
17 // In .h file:
18 virtual Sphere::print_5() = 0;
19
20
21
22
```

RedBall.cpp

```
1 // No print_1() in RedBall.cpp
2
3
4
5 RedBall::print_2() {
6     cout << "Ball" << endl;
7 }
8
9 // No print_3() in RedBall.cpp
10
11
12
13 RedBall::print_4() {
14     cout << "Ball" << endl;
15 }
16
17 RedBall::print_5() {
18     cout << "Ball" << endl;
19 }
20
21
22
```

Virtual Function

- A member function that you expect to be redefined in derived classes.
- When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the function.
- Pure virtual function
 - `virtual Sphere::print_5() = 0`
 - not implemented in the base class itself
 - implemented only in the derived classes

	<code>Sphere obj;</code>	<code>RedBall obj;</code>	<code>RedBall r; Sphere &obj = r;</code>
<code>obj.print_1();</code>	"Sphere"	<i>No print_1() is defined in RedBall, so we use the base class (Sphere)'s print_1(): "Sphere"</i>	"Sphere"
<code>obj.print_2();</code>	"Sphere"	<i>The type of <code>obj</code> is RedBall, so we'll use RedBall's implementation: "Ball"</i>	The type of <code>obj</code> is Sphere, so we'll use Sphere's impl since Sphere::print_2() <u>is not</u> virtual: "Sphere"
<code>obj.print_3();</code>	"Sphere"	<i>No print_3() is defined in RedBall, so we use the base class (Sphere)'s print_3(): "Sphere"</i>	"Sphere"
<code>obj.print_4();</code>	"Sphere"	<i>The type of <code>obj</code> is RedBall, so we'll use RedBall's implementation: "Ball"</i>	The type of <code>obj</code> is Sphere, but Sphere::print_4() <u>is virtual</u> . Therefore, we will use the derived class' impl: "Ball"
<code>obj.print_5();</code>	Will not compile since Sphere is an abstract class when print_5() is defined as a pure virtual function.	<i>The type of <code>obj</code> is RedBall, so we'll use RedBall's implementation: "Ball"</i>	The type of <code>obj</code> is Sphere, but Sphere::print_5() <u>is virtual</u> . Therefore, we will use the derived class' impl: "Ball"

Abstract Class:

[Usage]:

- used to express broad concepts from which more concrete classes can be derived

[Requirement]:

- contains one or more pure virtual functions

[Syntax]:

- to make a function() pure virtual, add =0 at the end, e.g.,
function()=0;

[As a result]:

- if a class has a pure virtual function, we cannot create an instance of it

derived-defaultCtor.cpp

```
1 class Sphere {
2     public:
3         Sphere(double d) { /* ... */ }
4
5
6 }
7
8 class Ball : public Sphere {
9
10 }
11
12 int main() {
13     Ball b;
14     return 0;
15 }
16
```


derived-defaultCtor.cpp

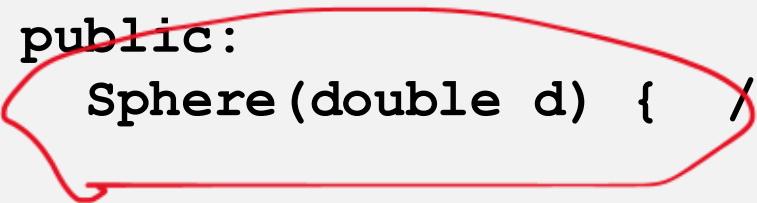
```
1 class Sphere {
2     public:
3         Sphere(double d) { /* ... */ }
4
5
6 }
7
8 class Ball : public Sphere {
9     Ball() : Sphere(1) { ... }
10 }
11
12 int main() {
13     Ball b;
14     return 0;
15 }
16
```

derived-defaultCtor.cpp

```
1 class Sphere {
2     public:
3         Sphere(double d) { /* ... */ }
4         Sphere () : Sphere(1.0) { ... }
5
6 }
7
8 class Ball : public Sphere {
9
10 }
11
12 int main() {
13     Ball b;
14     return 0;
15 }
16
```

derived-defaultCtor.cpp

```
1 class Sphere {  
2     public:  
3         Sphere(double d) { /* ... */ }  
4  
5  
6 }  
7  
8 class Ball : public Sphere {  
9  
10 }  
11  
12 int main() {  
13     Ball b(1.0);  
14     return 0;  
15 }  
16
```



custom constructor

virtual-ctor.cpp

```
15 class Sphere {  
16     public:  
17         virtual Sphere();  
18 }  
19  
20 class Ball : public Sphere {  
21     public:  
22         _____;  
23 }  
24
```

Constructors cannot be virtual

virtual-ctor.cpp

```
15 class Sphere {  
16     public:  
17     virtual Sphere();  
18 }  
19  
20 class Ball : public Sphere {  
21     public:  
22     _____;  
23 }  
24
```

Constructors cannot be virtual

The base class is going to have all the generic code

Otherwise, it will be overwritten by the derived class

virtual-dtor.cpp

```
15 class Sphere {  
16     public:  
17         virtual ~Sphere();  
18 }  
19  
20 class Ball : public Sphere {  
21     public:  
22         ~Ball();  
23 }  
24
```

should be virtual !

virtual-dtor.cpp

```
15 class Sphere {  
16     public:  
17         virtual ~Sphere();  
18 }  
19  
20 class Ball : public Sphere {  
21     public:  
22         ~Ball();  
23 }  
24
```

should be virtual !

Sphere *s = new Ball();

delete s;

Call Order – How are derived classes created?

- **base class' constructor is called first**
- **then derived class constructor is called**

Lab 3 example: drawable -> shape -> { circle, rectangle }

Call Order – How are derived classes destroyed?

- **In the reverse order**
- **Derived class' destructor is called first**
- **The most base class's destructor is called last**

sphere.cpp

```
10 void Sphere::destroy_() { delete[] props_; }
11
12 void Sphere::copy_(const Sphere &other) {
13     r_ = other.r;
14     props_max_ = other.props_max_;
15     props_ct_ = other.props_ct_;
16     props_ = new std::string[10];
17     for (unsigned i = 0; i < props_ct_; i++) {
18         props_[i] = other.props_[i];
19     }
20 }
21
22 Sphere& Sphere::operator=(const Sphere &other) {
23
24
25
26     destroy_();
27     copy_(other);
28     return *this;
29 }
```

If (this == &other) return *this

assignmentOpSelf.cpp

```
1 #include "Sphere.h"
2
3 int main() {
4     cs225::Sphere s(10);
5     s = s;
6     return 0;
7 }
```

Abstract Data Type (ADT)

- a model for a data type
- defined by its behavior
- from the point of view of the user

List ADT

- **add: we can add to it**
- **get: we can get an object from the list**
- **remove: we may want to remove an object from the list**
- **size: how many elements are in the list**
- **create an empty list: we have to start somewhere**
- **isEmpty: we may need to check if the list is empty or not**

What types of “stuff” do we want in our list?

1	2	3					
----------	----------	----------	--	--	--	--	--

int

a	b	c					
----------	----------	----------	--	--	--	--	--

float

1.2	2.3	3.4					
------------	------------	------------	--	--	--	--	--

double

A separate code for each data type ?

Templates

- **create functions with generic variables**
- **generic variables can have any type**

template1.cpp

```
1  Template <class T>  
2  
3  T maximum(T a, T b) {  
4      T result;  
5      result = (a > b) ? a : b;  
6      return result;  
7  }
```

template1.cpp

```
1  Template <class T>  
2  
3  T maximum(T a, T b) {  
4      T result;  
5      result = (a > b) ? a : b;  
6      return result;  
7  }
```

template2.cpp

```
1  Template <class T, class U>  
2  
3  T maximum(T a, U b) {  
4      T result;  
5      result = (a > b) ? a : b;  
6      return result;  
7  }
```


List.h

```
1 #ifndef LIST_H
2 #define LIST_H
3
4
5 template <class T>
6 class List {
7     public:
8         void add(const T& t)
9
10
11
12
13
14
15     private:
16
17
18
19 };
20
21 #endif
22
```

List.cpp

```
1
2
3 template <class T>
4 void List::add(const T& t) {
5
6     ...
7
8 }
9
10
11
12
13
14
15
```

ListMain.cpp

```
1 #include "List.h"
2
3 int main() {
4     List          lst;
5     lst.add(1);
6     return 0;
7 }
```

MP: Extra Credit

The most successful MP is an MP done early!

Unless otherwise specified in the MP, we will award +7 extra credit points if the first part of the MP is submitted by the EC deadline.

MP2 example:

MP2 Due Date: Monday, March **16**

MP2 EC Due Date: Monday, March **9**

MP1 and MP5 will not have EC

MP2, MP3, MP5, MP6, and MP7 will have EC

5 x 7 = 35 EC points!

CS 225 – Things To Be Doing

lab_inheritance is out

Due: Sunday, March 8 (11:59pm)

MP2 is out – Early Deadline Monday, March 9

Up to +7 Extra Credit for Early Submission